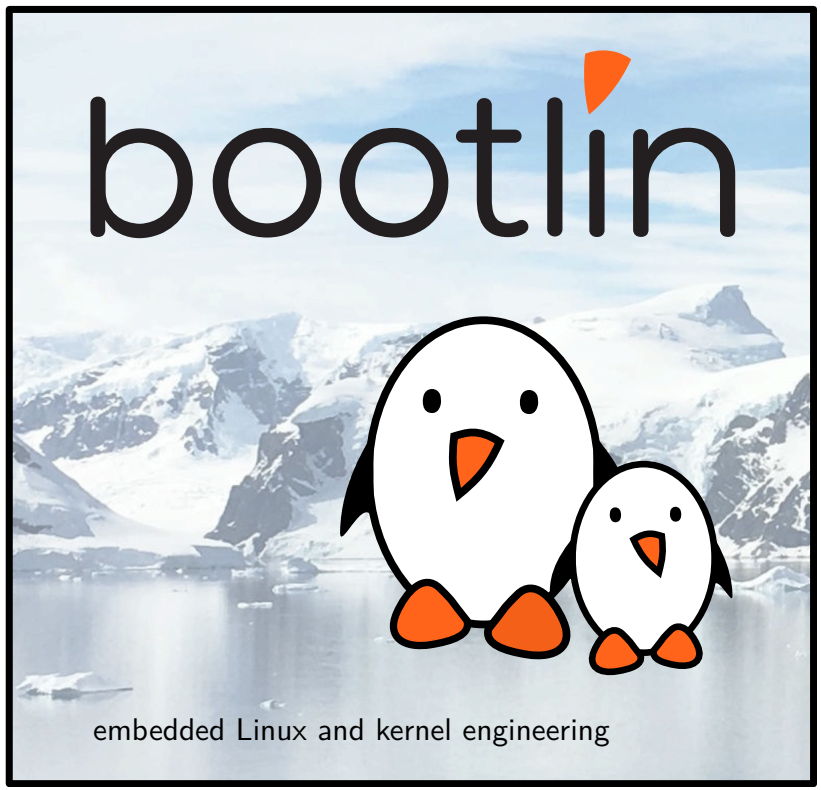
Linux debugging, profiling and tracing training

© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Latest update: November 08, 2026.

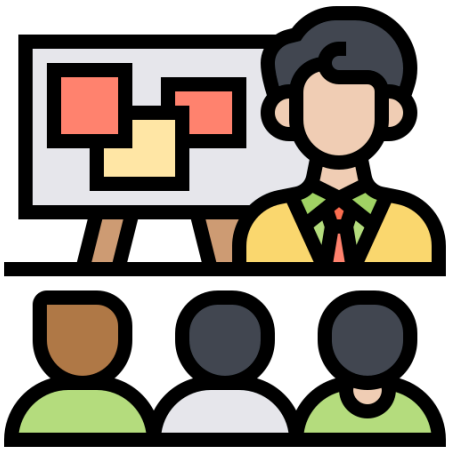
Document updates and training details:
https://bootlin.com/training/debugging

Corrections, suggestions, contributions and translations are welcome!
Send them to feedback@bootlin.com



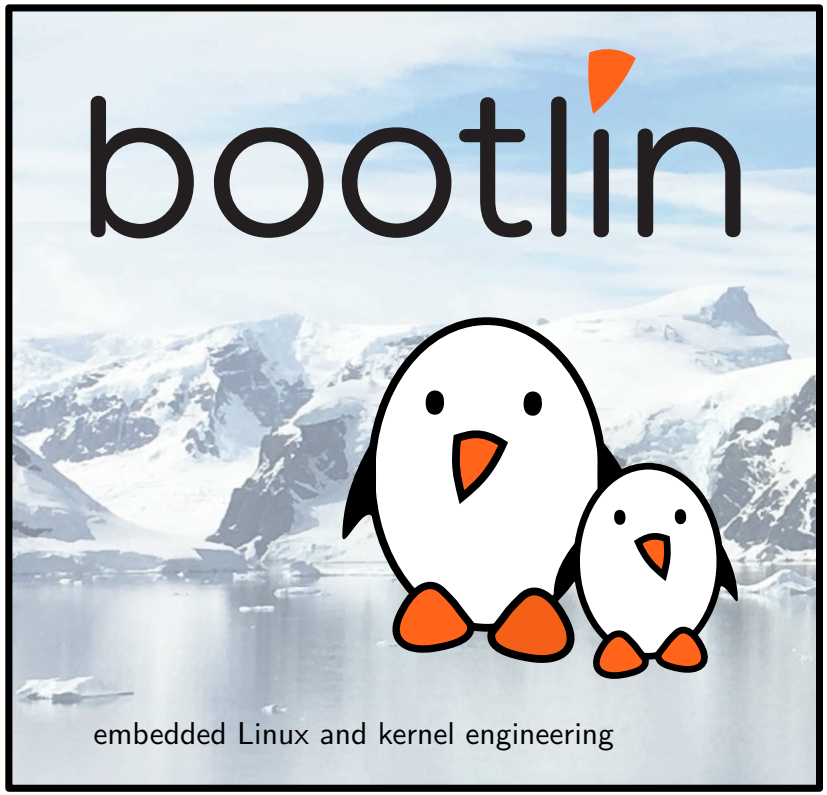
Linux debugging, profiling and tracing training

- These slides are the training materials for Bootlin's *Linux debugging, profiling and tracing* training course.
- If you are interested in following this course with an experienced Bootlin trainer, we offer:
  - **Public online sessions**, opened to individual registration. Dates announced on our site, registration directly online.
  - **Dedicated online sessions**, organized for a team of engineers from the same company at a date/time chosen by our customer.
  - **Dedicated on-site sessions**, organized for a team of engineers from the same company, we send a Bootlin trainer on-site to deliver the training.
- Details and registrations: https://bootlin.com/training/debugging
- Contact: `training@bootlin.com`



Icon by Eucalyp, Flaticon

About Bootlin



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Bootlin introduction

- Engineering company
  - In business since 2004
  - Before 2018: *Free Electrons*
- Team based in France and Italy
- Serving **customers worldwide**
- **Highly focused and recognized expertise**
  - Embedded Linux
  - Linux kernel
  - Embedded Linux build systems
- **Strong open-source** contributor
- Activities
  - **Engineering** services
  - **Training** courses
- https://bootlin.com

bootlin



Bootlin engineering services

Bootloader / firmware development

U-Boot, Barebox, OP-TEE, TF-A, .../

Linux kernel porting and driver development

Linux BSP development, maintenance and upgrade

Embedded Linux build systems

Yocto, OpenEmbedded, Buildroot, ...

Embedded Linux integration

Boot time, real-time, security, multimedia, networking

Open-source upstreaming

Get code integrated in upstream Linux, U-Boot, Yocto, Buildroot, ...



Bootlin training courses

Embedded Linux system development On-site: 4 or 5 days Online: 7 * 4 hours	Linux kernel driver development On-site: 5 days Online: 7 * 4 hours	Yocto Project system development On-site: 3 days Online: 4 * 4 hours	Buildroot system development On-site: 3 days Online: 5 * 4 hours	Embedded Linux networking On-site: 3 days Online: 4 * 4 hours
Understanding the Linux graphics stack On-site: 2 days Online: 4 * 4 hours	Embedded Linux audio On-site: 2 days Online: 4 * 4 hours	Embedded Linux Security On-site: 3 days Online: 4 * 4 hours	Linux debugging, tracing, profiling and performance analysis On-site: 3 days Online: 4 * 4 hours	Real-Time Linux with PREEMPT_RT On-site: 2 days Online: 3 * 4 hours

All our training materials are freely available
under a free documentation license (CC-BY-SA 3.0)
See https://bootlin.com/training/



Bootlin, an open-source contributor

- Strong contributor to the **Linux** kernel
  - In the top 30 of companies contributing to Linux worldwide
  - Contributions in most areas related to hardware support
  - Several engineers maintainers of subsystems/platforms
  - 9000 patches contributed
  - https://bootlin.com/community/contributions/kernel-contributions/
- Contributor to **Yocto Project**
  - Maintainer of the official documentation
  - Core participant to the QA effort
- Contributor to **Buildroot**
  - Co-maintainer
  - 6000 patches contributed
- Significant contributions to U-Boot, OP-TEE, Barebox, etc.
- Fully **open-source training materials**

Bootlin on-line resources

- Website with a technical blog:
  https://bootlin.com
- Engineering services:
  https://bootlin.com/engineering
- Training services:
  https://bootlin.com/training
- LinkedIn:
  https://www.linkedin.com/company/bootlin
- Elixir - browse Linux kernel sources on-line:
  https://elixir.bootlin.com



Icon by Freepik, Flaticon

Generic course information



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!

STM32MP157 shopping list

- Discovery Kits from STMicroelectronics: STM32MP157A-DK1, STM32MP157D-DK1, STM32MP157C-DK2 or STM32MP157F-DK2[1]
  - STM32MP157 (Dual Cortex-A7 + Cortex-M4) CPU
  - 512 MB DDR3L RAM
  - Plenty of peripherals: GPIOs, SPI, Serial, USB, Ethernet...
- MicroUSB cable (to access serial console)
- USB-C to USB-A cable (to power the board)
- MicroSD card
- RJ45 cable



[1] Boards documentation: A-DK1, D-DK1, C-DK2, F-DK2

BeaglePlay shopping list

- BeaglePlay, from BeagleBoard.org
  - Texas Instruments AM625x (4×ARM Cortex-A53 CPU)
  - 2 GB RAM
  - 16 GB of on-board eMMC storage
  - Peripherals: SPI, I2C, UART, USB...
- USB-C cable for the power supply
- A USB-FTDI cable
- RJ45 cable for networking
- A micro SD card with at least 2G of capacity





Training quiz and certificate

- To get your training certificate you must
  1. Attend all sessions of this training course
  2. Achieve more than 50% of correct answers at our final quiz
     - The final quiz questions are identical to the pre-training quiz
     - The final quiz must be completed within two weeks of the session end's date
- The training certificate will be sent to you two weeks after the session end's date.



Participate!

During the lectures...

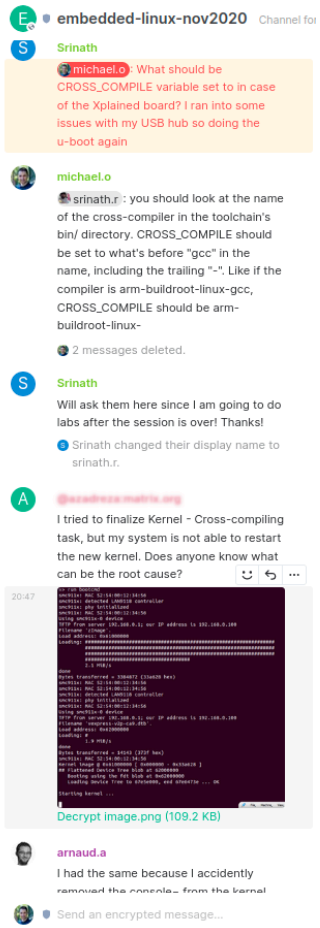
- Don't hesitate to ask questions. Other people in the audience may have similar questions too.
- Don't hesitate to share your experience too, for example to compare Linux with other operating systems you know.
- Your point of view is most valuable, because it can be similar to your colleagues' and different from the trainer's.
- In on-line sessions
  - Please always keep your camera on!
  - Also make sure your name is properly filled.
  - You can also use the "Raise your hand" button when you wish to ask a question but don't want to interrupt.
- All this helps the trainer to engage with participants, see when something needs clarifying and make the session more interactive, enjoyable and useful for everyone.



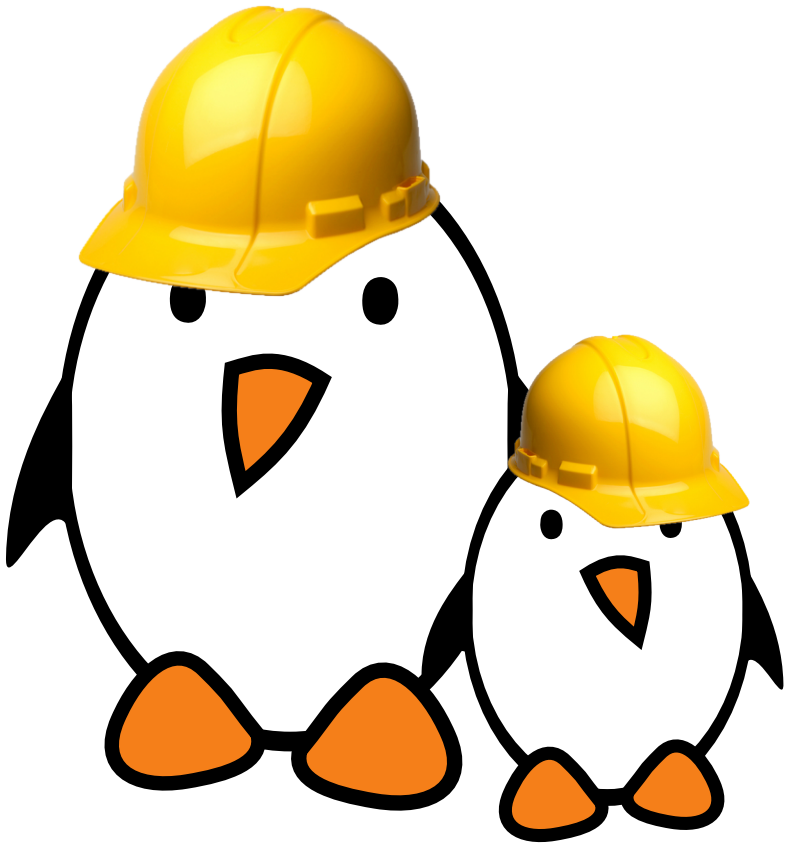
Collaborate!

As in the Free Software and Open Source community, collaboration between participants is valuable in this training session:

- Use the dedicated Matrix channel for this session to add questions.
- If your session offers practical labs, you can also report issues, share screenshots and command output there.
- Don't hesitate to share your own answers and to help others especially when the trainer is unavailable.
- The Matrix channel is also a good place to ask questions outside of training hours, and after the course is over.



Practical lab - Training Setup



Prepare your lab environment

- Download and extract the lab archive

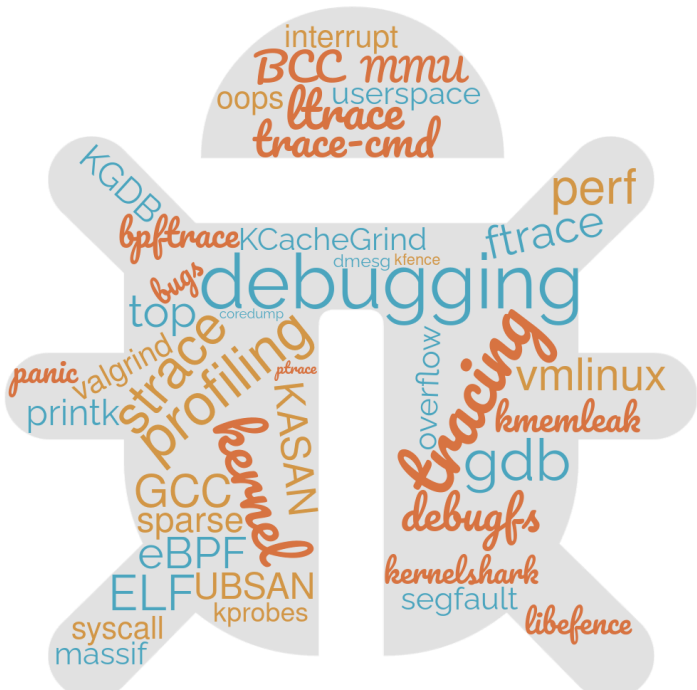
Debugging, Tracing, Profiling



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!

Debugging, Tracing, Profiling

- Debugging, tracing and profiling are often used for development purposes
- All of these methods have different goals which aim at perfecting the software that is being developed
- Requires some knowledge about underlying mechanisms to correctly identify and fix bugs





Debugging

- Finding and fixing bugs that might exist in your software/system
- Use of various tools and methods to achieve that
  - Interactive debugging (With GDB for instance)
  - Postmortem analysis (Using coredump for instance)
  - Control flow analysis (With tracing tools)
  - Testing (Targeted tests)
- Most commonly done through debuggers in development environment
- Generally intrusive, allowing to pause and resume execution

"Everyone knows that debugging is twice as hard as writing a program in the first place. So if you're as clever as you can be when you write it, how will you ever debug it?"
- Brian Kernighan



Tracing

- Following the execution flow of an application to understand the bottlenecks and problems.
- Achieved by instrumenting code either at compile time or runtime.
  - Can be done using specific tracers such as *LTTng*, *trace-cmd*, *SystemTap* etc
- Goes from the user space called functions up to the kernel ones
- Allows to identify functions and values that are used while application executes
- Often works by recording traces during runtime and then visualizing data.
  - Implies a large amount of recorded data since the complete execution trace is recorded
  - Often bigger overhead than profiling.
- Can also be used for debugging purpose since data can be extracted with tracepoints.



Profiling

- Analysis at program runtime to assist performance optimizations
- Often achieved by sampling counters during execution
- Uses specific tools, libraries and operating system features to measure performance.
  - Using *perf*, *OProfile* for instance.
- First step consists in gathering data from program execution
  - Function call count, memory usage, CPU load, cache miss, etc
- Then extracting meaningful information from these data and modify the program to optimize it



Event sources

- Some activities like tracing or profiling involve some preliminary data collection
- Those data are gathered from different **event sources**, which can be of various types.
  - Some low-level events are gathered directly by the hardware (eg: CPU cycles, MMU exceptions...)
  - Some events are generated by some code explicitly added to generate traces, either in an application or in the kernel: those are **static tracepoints**
  - Some are generated by instrumentation added at runtime (ie without having to modify/rebuild the application and/or the kernel): those are **dynamic probes**

Linux Application Stack

© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!





User/Kernel mode



User/Kernel mode

- User mode vs Kernel mode are often used to refer to the privilege level of execution.
- This mode actually refers to the processor execution mode which is a hardware mode.
  - Might be named differently between architectures but the goal is the same
- Allows the kernel to control the full processor state (handle exceptions, MMU, etc) whereas the userspace can only do basic control and execute under the kernel supervision.



Introduction to Processes and Threads



Processes and Threads (1/2)

- A process is a group of resources that are allocated by the kernel to allow the execution of a program.
  - Memory regions, threads, file descriptors, etc.
- A process is identified by a PID (**P**rocess **ID**) and all the information that are specific to this process are exposed in `/proc/<pid>`.
  - A special file named `/proc/self` accessible by the process points to the proc folder associated to it.
- When starting a process, it initially has one execution thread that is represented by a struct task_struct and that can be scheduled.
  - A process is represented in the kernel by a thread associated to multiple resources.



Processes and Threads (2/2)

- Threads are independent execution units that are sharing common resources inside a process.
  - Same address space, file descriptors, etc.
- A new process is created using the fork() system call (man 2 fork) and a new thread is created using `pthread_create()` (man 3 pthread_create).
  - Internally, both will call `clone()` with different flags
- At any moment, only one task is executing on a CPU core and is accessible using get_current() function (defined by architecture and often stored in a register).
- Each CPU core will execute a different task.
- A task can only be executing on one core at a time.

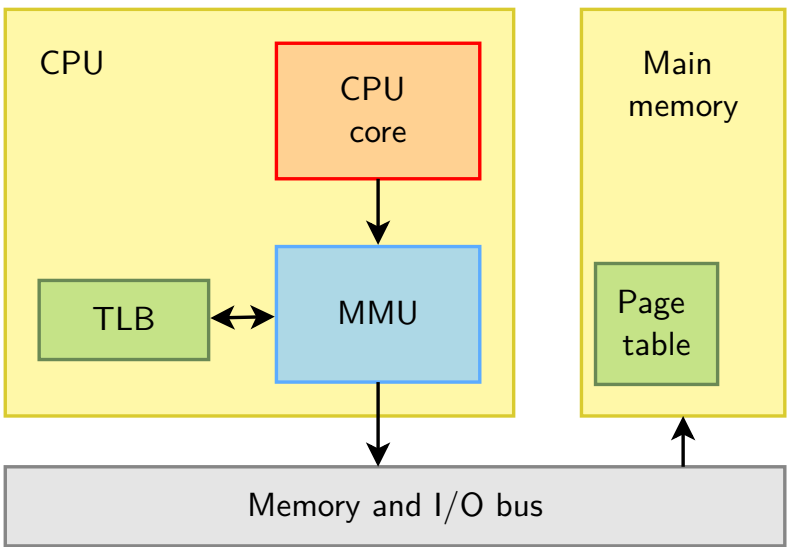


MMU and memory management



The MMU

- Addresses accessed by the CPU are *virtual*
- The *Memory Management Unit* (MMU) translates them to *physical* addresses
- The kernel decides the translation and fills the *page table*
- The MMU reads the page table and caches recent entries in the *Translation lookaside buffers* (TLB) for zero-delay mapping





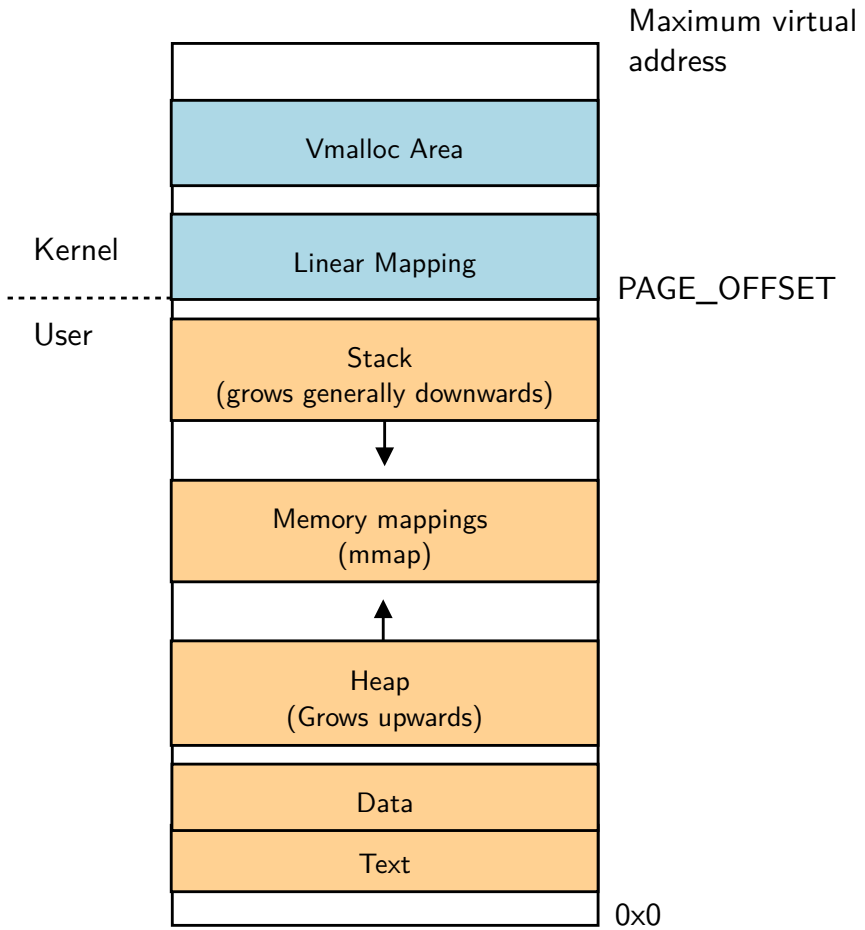
MMU and memory management

- Physical addresses can be either RAM or I/O (to access devices)
- The MMU allows to restrict access to the page mappings via some attributes
  - No Execute, Writable, Readable bits, Privileged/User bit, cacheability
- The MMU base unit for mappings is called a page
- Page size is fixed and depends on the architecture/kernel configuration.
- Linux can work without an MMU (CONFIG_MMU=n), useful for old SoCs without an MMU, but with many limitations



Userspace/Kernel memory layout

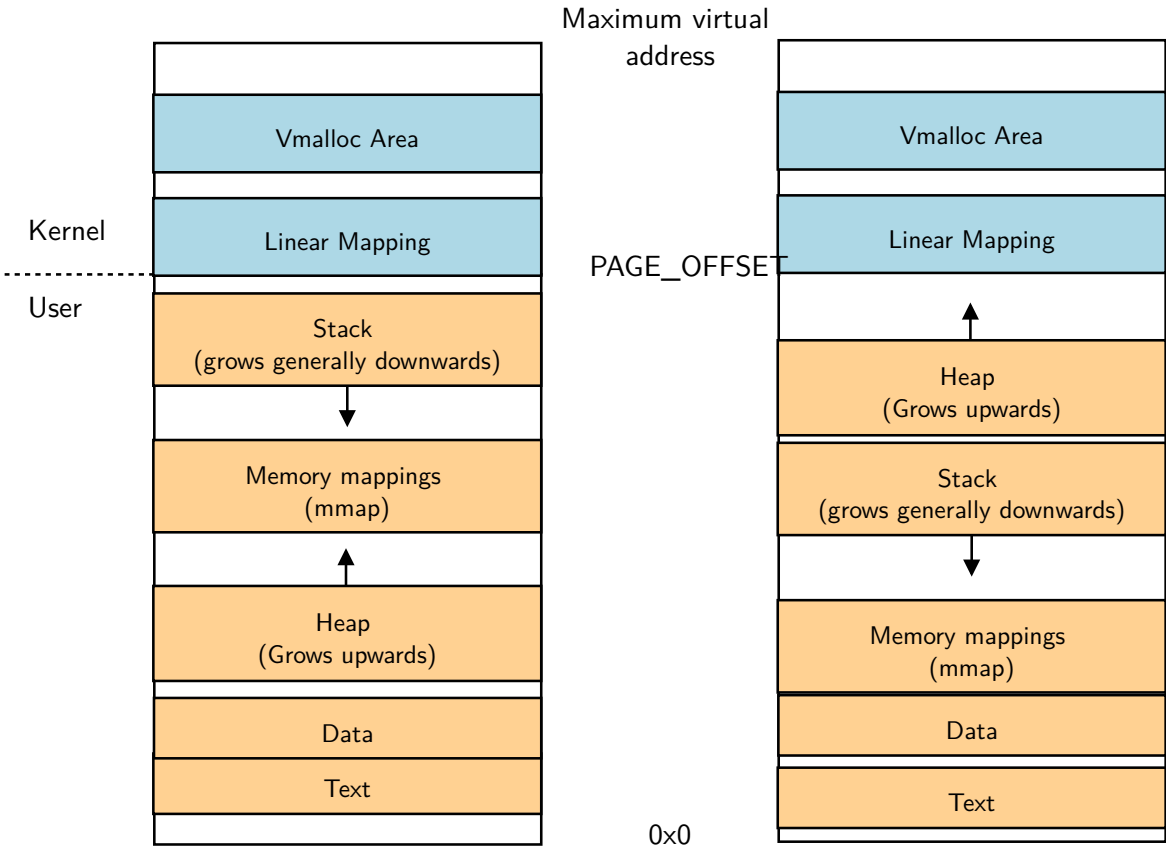
- Each process has its own set of virtual memory areas (`mm` field of struct task_struct).
- Also have their own page table
  - But share the same kernel mappings
- By default, all user mapping addresses are randomized to minimize attack surface (base of heap, stack, text, data, etc).
  - **A**ddress **S**pace **L**ayout **R**andomization
  - Can be disabled using `norandmaps` command line parameter





Userspace/Kernel memory layout

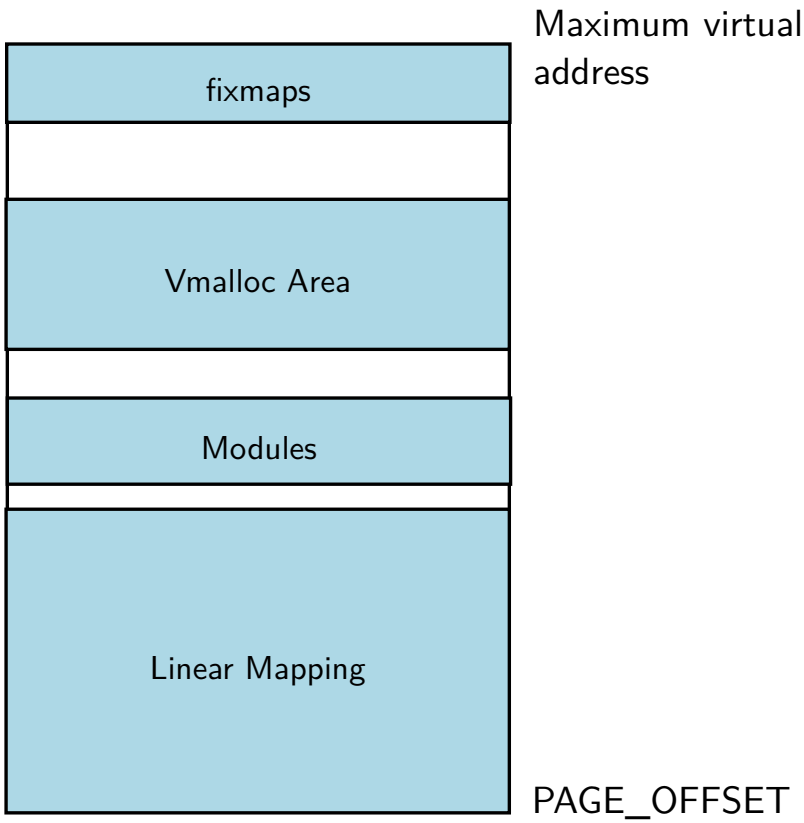
Multiple processes have different user memory spaces





Kernel memory map

- The kernel has it own memory mapping.
- Linear mapping is setup at kernel startup by inserting all the entries in the kernel init page table.
- Multiple areas are identified and their location differs between the architectures.
- **K**ernel **A**ddress **S**pace **L**ayout **R**andomization also allows to randomize kernel address space layout.
  - Can be disabled using `nokaslr` command line parameter





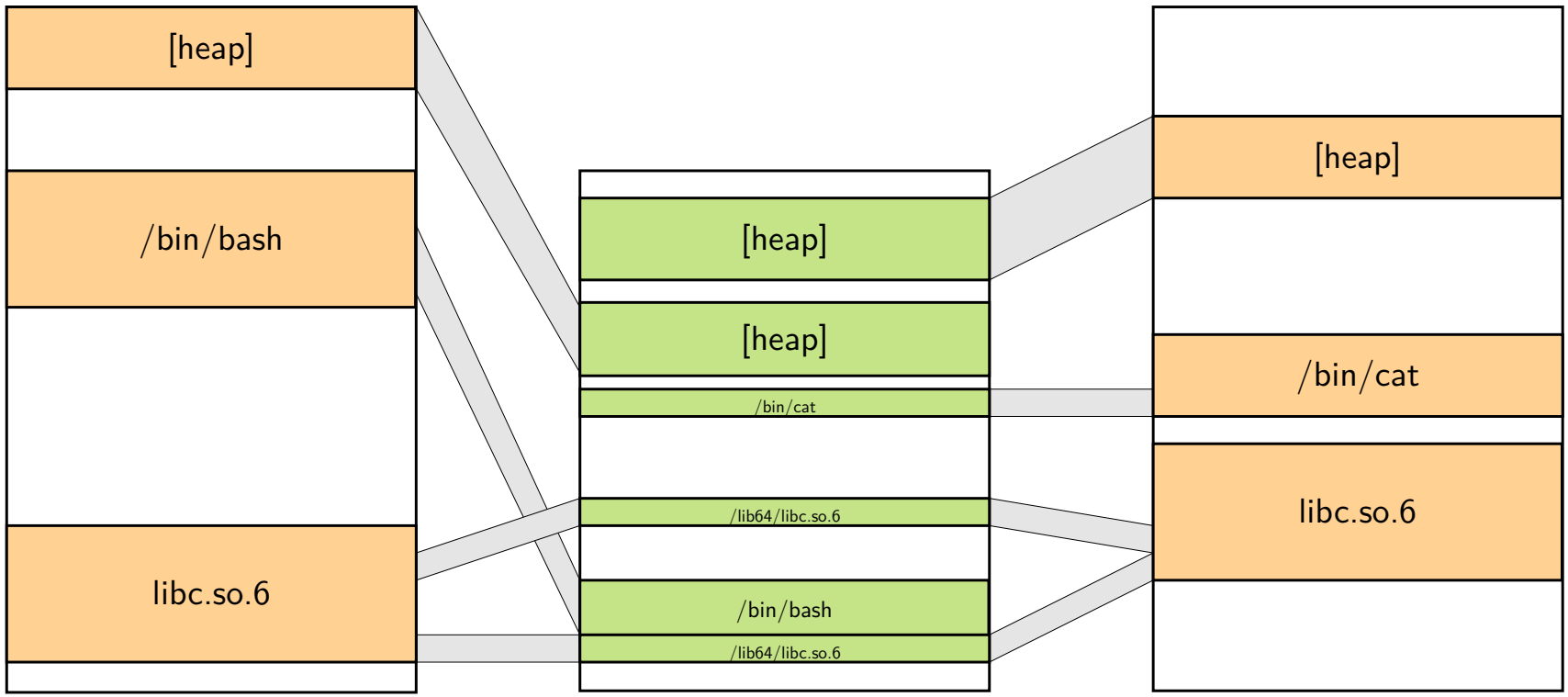
Userspace memory segments

- When starting a process, the kernel sets up several *Virtual Memory Area*s (VMA), backed by struct vm_area_struct, with different execution attributes.
- VMA are actually memory zones that are mapped with specific attributes (R/W/X).
- A segmentation fault happens when a program tries to access an unmapped area or a mapped area with an access mode that is not allowed.
  - Writing data in a read-only segment
  - Executing data from a non-executable segment
- New memory zones can be created using `mmap()` (man 2 mmap)
- Per application mappings are visible in `/proc/<pid>/maps`

```
7f1855b2a000-7f1855b2c000 rw-p 00030000 103:01 3408650  ld-2.33.so
7ffc01625000-7ffc01646000 rw-p 00000000 00:00 0         [stack]
7ffc016e5000-7ffc016e9000 r--p 00000000 00:00 0         [vvar]
7ffc016e9000-7ffc016eb000 r-xp 00000000 00:00 0         [vdso]
```

Virtual memory VS physical memory

- Memory segments can be shared among different processes
- Non-contiguous physical memory can be virtually contiguous





Userspace memory types

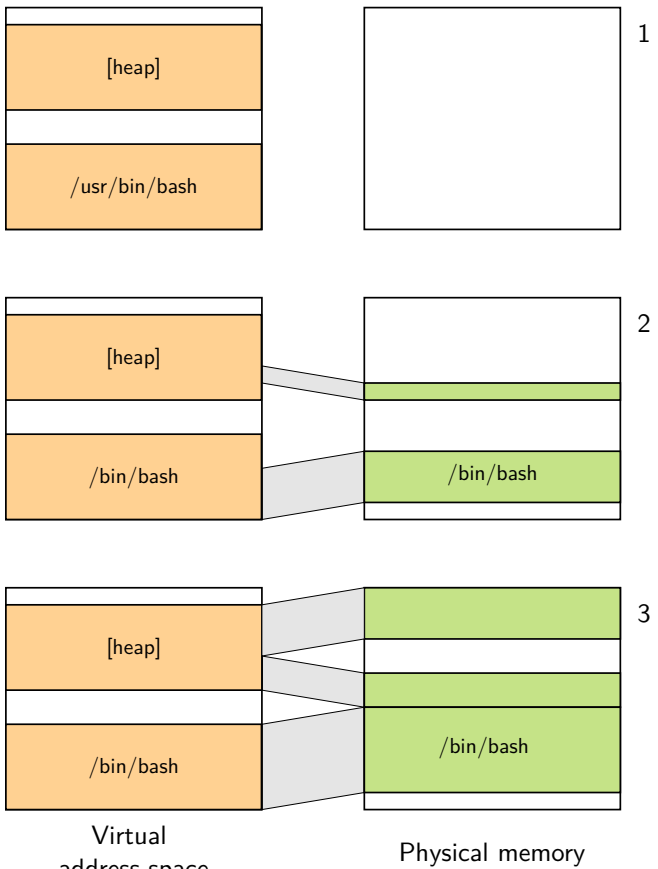
	Private	Shared
Anonymous	- stack - malloc() - brk()/sbrk() - mmap(PRIVATE,ANON)	- POSIX shm* - mmap(SHARED, ANON)
File-backed	- mmap(PRIVATE,fd) - program (code) - shared libraries	- mmap(SHARED, fd)



On-demand memory mapping (*Lazy allocation*)

1. Virtual memory is allocated when requested, but not mapped to physical memory
2. When the CPU accesses a virtual address not yet physically mapped, a *page fault* happens, and the kernel physically allocates memory
   - For file-backed mappings, implies reading from disk
3. The virtual memory is fully mapped to physical memory only if really needed

- Allows faster program startup, avoids using memory for unused data
- Execution time not deterministic: for real-time needs, memory can be *pre-faulted*





On-demand memory mapping: page faults

- Lazy allocation is implemented based on *Page faults*
  1. The CPU accesses a virtual address, valid but not yet physically mapped
     - E.g.: executes an instruction or reads/writes data
  2. The MMU finds no mapping in the Page table → issues a Page fault *exception* to the CPU
  3. The CPU is immediately turned in kernel mode and executes an exception vector (a function provided by the kernel)
  4. The kernel checks if the virtual address is valid, finds a suitable physical page to map, fills the page (reads from disk if needed) and creates a page table entry
  5. The kernel continues execution normally
  6. The MMU now finds a mapping and continues



Terms for memory in Linux tools

- When using Linux tools, four terms are used to describe memory:
  - *VSS/VSZ*: Virtual Set Size (Virtual memory size, shared libraries included).
  - *RSS*: Resident Set Size (Total physical memory usage, shared libraries included).
  - *PSS*: Proportional Set Size (Actual physical memory used, divided by the number of times it has been mapped).
  - *USS*: Unique Set Size (Physical memory occupied by the process, shared mappings memory excluded).
- VSS >= RSS >= PSS >= USS.